

Web Parsing Pipeline

Laboratorio di Ingegneria Informatica

Tommaso Boi
Matricola: 2103837

Dalila Maria Guardiani
Matricola: 1991867

Alessandro Buoncompagni
Matricola: 2128727

guardiani.1991867@studenti.uniroma1.it

boi.2103837@studenti.uniroma1.it

buoncompagni.2128727@studenti.uniroma1.it

1 Obiettivo del progetto

Abbiamo realizzato una pipeline che prende delle pagine web da domini diversi e ne tira fuori il testo pulito in Markdown. Il tutto è esposto tramite una REST API (FastAPI) con una piccola interfaccia web (Jinja2), si appoggia a un database MariaDB e a un LLM-as-Judge (Ollama), e gira dentro Docker Compose. I quattro domini gestiti sono *it.wikipedia.org*, *news.microsoft.com*, *pmc.ncbi.nlm.nih.gov* e *it.investing.com*.

2 Organizzazione del codice

La struttura segue quella suggerita nelle slide.

2.1 Backend (backend/src/)

- **server.py**: il web server FastAPI con tutti gli endpoint (*/parse*, */domains*, */gold_standard*, */evaluate*, */evaluate_judge*, */full_gs_eval*, */db_stats*, ecc.). In base al dominio dell'URL sceglie il parser giusto tramite un registro (factory), senza catene di *if/elif*.
- **parser_base.py**: la classe astratta *BaseParser* e il meccanismo di registrazione dei parser per dominio.
- **parser_wikipedia.py**: usa *Crawl4AI* in modalità headless. Poi con un po' di regex togliamo info-box, tabelle finali e template di avviso.
- **parser_microsoft.py**: qui il problema era Cloudflare, quindi scarichiamo l'HTML grezzo e lavoriamo di *BeautifulSoup* per prendere solo titolo e corpo.
- **parser_pubmed.py**: anche qui l'HTML pubblico dava problemi, così invece di scrapare l'HTML usiamo le *NCBI E-utilities (efetch)*, che restituiscono direttamente il JATS XML. Poi lo parsiamo sempre con *BeautifulSoup*.

- **parser_investing.py**: anche questo dominio è dietro Cloudflare con challenge JavaScript, quindi scarichiamo la pagina renderizzata con *Crawl4AI* e poi estraiamo il contenuto con *BeautifulSoup*. Il parser distingue le pagine editoriali (news/analisi), le schede strumento (valute, indici) e le pagine di tipo listing/calendario, e scarta i widget di contorno (grafici, pannelli "panoramica mercati").
- **evaluation.py**: le tre metriche di valutazione — *token_level_eval*, *char_overlap_eval* e *bleu1_eval*.
- **llm_judge.py**: la valutazione LLM-as-Judge tramite Ollama.
- **database.py** e **init_db.py**: connessione a MariaDB, creazione dello schema e popolazione iniziale dal GS.
- **gold_standard.py**: carica i file JSON del GS dentro *gs_data/*.

2.2 Frontend

frontend.py espone l'interfaccia sulla porta 8004: si può incollare un URL e vedere a schermo il testo parsato, l'HTML grezzo e, se l'URL è nel GS, anche il confronto e le metriche. Il tutto è orchestrato da **docker-compose.yaml**.

3 Valutazione dei parser

3.1 Metodo

Per valutare i parser abbiamo costruito un Gold Standard con 10 pagine per dominio, scegliendole apposta di tipologie diverse per non farci ingannare da un parser che funziona bene solo su un tipo di pagina.

La metrica principale è la *token_level_eval*: splitiamo *parsed* e *gold* in token lowercase (dopo aver tolto il Markdown), e calcoliamo Precision, Recall e F1 sull'intersezione dei due insiemi. Oltre

a quella, abbiamo voluto aggiungere due metriche complementari per avere un quadro più completo.

La prima è *char_overlap_eval*: è più tollerante a differenze di spaziatura e punteggiatura, in pratica ci dice se il testo è corretto a livello di caratteri.

La seconda è *bleu1_eval*: qui teniamo conto delle ripetizioni e penalizziamo se l’output è molto più corto del GS. Serve soprattutto per trovare i parser che tagliano via troppi contenuti.

3.2 Risultati

I risultati sulla metrica principale sono in Tabella 1: F1 fra 0.94 e 0.98. Usando la scala delle slide, con l’aggiunta di una fascia “Ottimo” per $F1 > 0.95$, Microsoft finisce in **Ottimo**, mentre Wikipedia, PubMed e Investing restano in **Buono**. Nessun parser va sotto lo 0.80.

La Tabella 2 mostra le metriche extra. Sono abbastanza in linea con la principale, ma ci hanno aiutato a interpretare meglio gli errori.

La *bleu1_eval* è sempre più bassa perché considera anche le ripetizioni e penalizza quando l’output è corto. Anche qui Microsoft è il più solido (0.96). In sintesi, le tre metriche insieme ci permettono di capire che tipo di errore stiamo facendo.

Dominio	P	R	F1
it.wikipedia.org	0.93	0.98	0.95
news.microsoft.com	0.97	1.00	0.98
pmc.ncbi.nlm.nih.gov	0.91	0.97	0.94
it.investing.com	0.93	0.96	0.94

Table 1: P, R e F1 medi per dominio (*token_level_eval*).

Dominio	char P	char F1	BLEU-1
it.wikipedia.org	0.93	0.95	0.90
news.microsoft.com	0.97	0.98	0.96
pmc.ncbi.nlm.nih.gov	0.92	0.95	0.88
it.investing.com	0.92	0.94	0.88

Table 2: Metriche complementari: *char_overlap_eval* e *bleu1_eval*.

4 Valutazione del Judge

Come valutazione alternativa abbiamo aggiunto un LLM-as-Judge basato su Ollama, con il modello `llama3.2:3b`, scelto perché dà output JSON affidabili e gira bene su CPU. Al modello passiamo il testo parsato e il gold di riferimento (troncati per restare entro i limiti di contesto) e gli chiediamo di

restituire un JSON con un punteggio intero da 1 a 5 e un breve feedback testuale.

Per robustezza, qualsiasi problema (Ollama non raggiungibile, modello non ancora scaricato, JSON malformato) viene gestito da un *fallback*: in questi casi la funzione restituisce comunque un risultato valido con punteggio 1 e un feedback che spiega l’errore, così l’endpoint */evaluate_judge* non va mai in crash. I risultati del judge sono coerenti con le metriche automatiche: i domini con F1 più alto ricevono punteggi più alti.

5 Organizzazione del database

Usiamo MariaDB tramite il connector ufficiale *mysqlclient* per Python. Lo schema, creato automaticamente all’avvio, comprende quattro tabelle. **web_resources** contiene le risorse scaricate (*url* come chiave primaria, *domain*, *title*, *html_text*). **gold_standard** contiene il testo di riferimento (*url* come chiave primaria e al contempo chiave esterna verso *web_resources*, più *gold_text*): la relazione è quindi 1-a-1. **evaluations** e **judge_evaluations** memorizzano rispettivamente le metriche automatiche (token, char, BLEU-1) e l’esito del judge, ciascuna legata a *web_resources* tramite chiave esterna. Tutte le chiavi esterne usano *ON DELETE CASCADE*, così cancellando una risorsa si rimuovono automaticamente gold, metriche e giudizi collegati, mantenendo il database consistente.